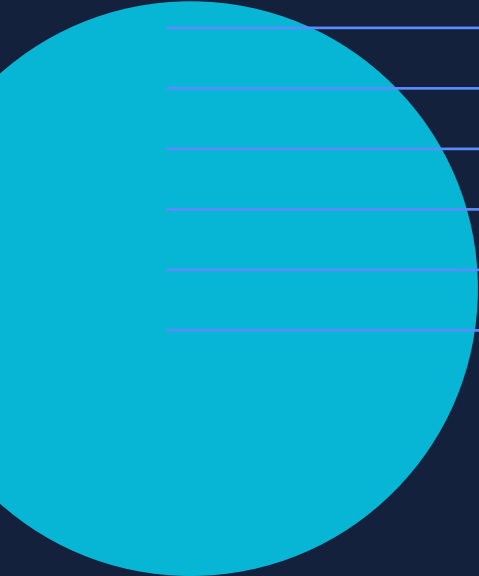




Spring Boot Mastery with Codex

A practical system for deep understanding, muscle memory,
debugging skill, and interview readiness

LEARN - BUILD - TEST - EXPLAIN - REBUILD



The Core Learning System

Codex is most valuable as a teacher, reviewer, interviewer, and debugging partner - not as a machine that writes every line for you.

The mastery loop

1	Learn	Build a clear mental model of one small concept.
2	Write	Implement it yourself without copying.
3	Test	Prove behavior with focused tests.
4	Debug	Investigate failures before requesting the answer.
5	Explain	Teach the concept back in your own words.
6	Rebuild	Recreate it later from requirements only.

The rule that changes everything

Retrieval creates durable knowledge. Reading an explanation feels smooth, but attempting to reconstruct it from memory creates the useful struggle that strengthens recall.

- Ask for hints before solutions.
- Predict program and database behavior before running the code.
- Keep a record of mistakes, incorrect assumptions, and corrected mental models.
- Revisit the same pattern in a different business domain.

Spring Boot Learning Roadmap

01	Java foundations OOP, collections, exceptions, streams, generics
02	Spring Core Beans, dependency injection, component scanning
03	Web APIs Controllers, DTOs, validation, HTTP semantics
04	Architecture Controller -> service -> repository boundaries
05	JPA & Hibernate Entities, relationships, fetching, transactions, queries
06	Error handling Domain exceptions and @ControllerAdvice
07	Testing JUnit, Mockito, MockMvc, integration tests
08	Security Authentication, JWT, roles, authorization
09	Production Profiles, logging, migrations, configuration
10	Advanced Redis, Kafka, Docker, observability

Sequence matters

Do not rush into Kafka or microservices before CRUD, JPA, transactions, testing, and security feel natural. Advanced infrastructure amplifies weak fundamentals rather than repairing them.

The Daily Muscle-Memory Session

10 min	Recall	Write what you remember before opening notes.
20 min	Learn	Study one narrow concept and its mental model.
35 min	Build	Implement a small behavior yourself.
20 min	Test & debug	Predict, run, inspect, and correct.
10 min	Explain	Teach it back and answer follow-up questions.
5 min	Log	Record one mistake and one durable insight.

1. Learn one concept

Keep the scope narrow: one annotation, one relationship, one transaction boundary, or one security rule.

Teach me @ManyToOne using Restaurant and Branch. Explain the database result, owning side, foreign key, fetching, cascade choices, and common mistakes.

2. Write from memory

Close the explanation and implement without copying. The moment of trying to recall is the exercise.

3. Request review - not replacement

Ask Codex to inspect your attempt and identify conceptual gaps while leaving the files unchanged.

Read my Branch entity. Do not modify it. Find conceptual errors and ask questions that help me fix them.

4. Predict before running

Predict tables, foreign keys, SQL, validation failures, and HTTP responses. Then compare the prediction with reality.

Debugging as Deliberate Practice

A failure is not an interruption to learning. It is the most information-rich part of the session.

Use the hint ladder

Prevent Codex from revealing too much too early. Escalate help gradually:

- Level 1: Ask one diagnostic question.
- Level 2: Ask which layer is probably responsible.
- Level 3: Ask for a relevant concept or documentation clue.
- Level 4: Ask for a pseudocode correction.
- Level 5: Request a complete solution only after explaining your failed attempts.

My test failed with this error. Do not give the solution immediately. Give me one diagnostic question or hint at a time.

A repeatable debugging checklist

Move through the system in order rather than changing random annotations:

- Restate the expected behavior and the actual behavior.
- Read the first meaningful exception and its root cause.
- Locate the failing boundary: HTTP, validation, security, service, JPA, or database.
- Form one hypothesis and identify evidence that would disprove it.
- Make the smallest correction and rerun the narrowest relevant test.
- Explain why the correction works; do not stop at a green test.

Mistake journal template

Use a compact record after difficult bugs:

Problem:
My assumption:
Actual cause:
Evidence that revealed it:
Correct mental model:
How I will recognize it next time:

The Three-Pass Mastery System

PASS 1	Guided	Codex explains each component while you implement. Focus on vocabulary and mental models.
PASS 2	Requirements only	Codex supplies behavior and acceptance criteria - but no code. You choose the structure.
PASS 3	Timed rebuild	Implement a smaller version without help, then request a strict review.

Example progression

Use related but different features so knowledge transfers instead of becoming memorized syntax:

- Pass 1: Build Restaurant CRUD with guidance.
- Pass 2: Build Branch CRUD from requirements.
- Pass 3: Build Table management alone in 45 minutes.

How Codex should test you

Rotate exercise formats to expose different kinds of understanding:

Completion	Fill missing annotations or methods.
Diagnosis	Find the defect in broken Spring Boot code.
Design	Turn a requirement into entities and endpoints.
Interview	Answer follow-ups until the explanation is precise.
Incident	Investigate a production-style transaction or security bug.

Projects That Build Transferable Skill

Project	Primary skill	Stretch goal
Todo API	CRUD and validation	Pagination and filtering
Library API	JPA relationships	Borrowing transactions
Expense Tracker	Queries and reporting	Date-range aggregation
Authentication API	JWT and roles	Refresh tokens
Inventory API	Concurrency	Optimistic locking
Product Catalogue	Redis caching	Cache invalidation
Notification Service	Kafka events	Retries and idempotency

Why several projects?

One large project can hide weak areas because familiar code becomes a crutch. Rebuilding the same architectural pattern in a new domain proves that the skill transfers.

SmartServe remains the main portfolio project. Small side projects are training grounds, not competing products.

A useful completion standard

A module is learned when you can design it, implement it, test it, debug it, and explain its tradeoffs without relying on a tutorial - not merely when the application starts.

Weekly Training Rhythm

Day	Focus	Deliverable
Mon	Mental model + guided build	One narrow vertical slice
Tue	Independent implementation	Working endpoint and validation
Wed	Relationships / transactions	Database behavior predictions
Thu	Tests and debugging	Failure investigation notes
Fri	Requirements-only rebuild	Feature recreated without code help
Sat	Interview and refactor	Precise explanations and cleaner design
Sun	Review and spaced recall	Flashcards and mistake journal

Spaced repetition schedule

Rebuild or quiz the concept after roughly 1 day, 3 days, 7 days, and 21 days. Increase the interval when recall is strong; shorten it when you need substantial help.

- Review mistakes, not every line of your notes.
- Mix old topics into new tasks instead of studying one category for weeks.
- Measure independence: how much help was needed, and at what level of the hint ladder?

Reusable Codex Prompts

Teacher mode

Teach me [TOPIC] at beginner-to-job-ready level. Explain one small concept at a time. Give me an exercise, make me predict behavior, and ask me to explain it back.

Review mode

Read my implementation. Do not modify files. Identify conceptual errors, security risks, missing tests, and unclear naming. Give hints before fixes.

Debug mode

Here is the failure. Do not reveal the solution. Ask one diagnostic question at a time and challenge unsupported assumptions.

Requirements mode

Give me acceptance criteria for [FEATURE] without code or architecture. After I design it, critique my decisions and tradeoffs.

Interview mode

Interview me about [TOPIC]. Ask one question at a time, follow vague answers, and finish with a score and revision topics.

Recall mode

Quiz me on [TOPIC] without multiple choice. Start with fundamentals, then code prediction, debugging, and design tradeoffs.

Your Standing Learning Contract

Be my Spring Boot teacher and code reviewer.

I will write all production code. Do not edit my files unless I explicitly request it.

For each topic:

1. Inspect my current project.
2. Explain the mental model.
3. Give me one small implementation task.
4. Make me predict the behavior.
5. Review my attempt.
6. Give hints before solutions.
7. Explain alternatives and tradeoffs.
8. Test me with debugging and interview questions.
9. Ask me to rebuild the concept later from memory.
10. Track concepts I understand and concepts needing revision.

Never let me copy code without understanding why every important annotation, method, and architectural decision exists.

The advice worth remembering

Tutorial completion feels like progress, but retrieval is progress. Repeatedly building, forgetting, struggling, debugging, and rebuilding is not evidence that learning is failing - it is the mechanism by which durable skill is formed.

Build less at once. Understand more deeply. Return to it until it is yours.